

WEEK-7

Task-1: To parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

Code:

```
GNU nano 8.7.1          input_redirect.c *
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd;
    pid_t pid;

    fd = open("input.txt", O_RDONLY);

    if (fd == -1) {
        perror("input.txt");
        return 1;
    }

    pid = fork();

    if (pid == 0) {
        dup2(fd, STDIN_FILENO);
        close(fd);

        execlp("sort", "sort", NULL);

        perror("sort");
        exit(1);
    }

    close(fd);
    waitpid(pid, NULL, 0);

    printf("Input redirection completed.\n");

    return 0;
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

Output:

```
root@Mani:~/skill$ nano input.txt
root@Mani:~/skill$ gcc input_redirect.c -o input_redirect
root@Mani:~/skill$ ./input_redirect
apple
banana
grape
orange
Input redirection completed.
root@Mani:~/skill$
```

Report: The program parses input redirection and opens the specified file in read-only mode.

The file descriptor is redirected to standard input using dup2().

Missing files are detected and appropriate error messages are displayed.

The program was tested successfully with valid and missing input files.

Task-2: To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

Code:

```
GNU nano 8.7.1          output_redirect.c *
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd;
    pid_t pid;

    fd = open("output.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

    if (fd == -1) {
        perror("output.txt");
        return 1;
    }

    pid = fork();

    if (pid == 0) {
        dup2(fd, STDOUT_FILENO);
        close(fd);

        execlp("ls", "ls", NULL);

        perror("ls");
        exit(1);
    }

    close(fd);
    waitpid(pid, NULL, 0);

    printf("Output redirected to output.txt\n");

    return 0;
}
```

 ^G Help	 ^O Write Out	 ^F Where Is	 ^K Cut	 ^T Execute
 ^X Exit	 ^R Read File	 ^\ Replace	 ^U Paste	 ^J Justify

Output:

```
Input redirection completed.  
root@Mani:~/skill$ nano output_redirect.c  
root@Mani:~/skill$ gcc output_redirect.c -o output_redirect  
root@Mani:~/skill$ ./output_redirect  
Output redirected to output.txt  
root@Mani:~/skill$ cat output.txt  
cd_command  
cd_command.c  
export_command  
export_command.c  
history  
history.c  
input.txt  
input_redirect  
input_redirect.c  
mini_shell  
mini_shell.c  
multiple_pipes  
multiple_pipes.c  
output.txt  
output_redirect  
output_redirect.c  
pipe  
pipe.c  
pipeline_structure  
pipeline_structure.c  
pwd_exit  
pwd_exit.c  
root@Mani:~/skill$
```

Report: The program parses output redirection and creates an output file using open().

Standard output is redirected to the file using dup2().

File creation and permission errors are handled appropriately.

The output file was verified successfully to contain the redirected command output.

Task-3: To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

Code:

```
GNU nano 8.7.1                                append_redirect.c *
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd;
    pid_t pid;

    fd = open("append.txt",
              O_WRONLY | O_CREAT | O_APPEND,
              0644);

    if (fd == -1) {
        perror("append.txt");
        return 1;
    }

    pid = fork();

    if (pid == 0) {
        dup2(fd, STDOUT_FILENO);
        close(fd);

        execlp("date", "date", NULL);

        perror("date");
        exit(1);
    }

    close(fd);
    waitpid(pid, NULL, 0);

    printf("Data appended successfully.\n");

    return 0;
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

Output:

```
root@Mani:~/skill$ nano append_redirect.c
root@Mani:~/skill$ gcc append_redirect.c -o append_redirect
root@Mani:~/skill$ ./append_redirect
Data appended successfully.
root@Mani:~/skill$ cat append.txt
Fri Sep 18 14:44:49 UTC 2026
root@Mani:~/skill$ ./append_redirect
Data appended successfully.
root@Mani:~/skill$
```

Report: The program parses the append operator and opens the file using O_APPEND.

Standard output is redirected to the file without removing existing contents.

New data is added at the end of the file and file errors are handled.

The program was tested successfully by performing multiple append operations.

Task-4: TO parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

Code:

```
GNU nano 8.7.1                                stderr_redirect.c *
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd;
    pid_t pid;

    fd = open("error.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

    if (fd == -1) {
        perror("error.txt");
        return 1;
    }

    pid = fork();

    if (pid == 0) {
        dup2(fd, STDERR_FILENO);
        close(fd);

        execlp("ls", "ls", "nonexistent_file", NULL);

        perror("ls");
        exit(1);
    }

    close(fd);
    waitpid(pid, NULL, 0);

    printf("Error output redirected to error.txt\n");

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Output:

```
root@Mani:~/skill$ nano stderr_redirect.c
root@Mani:~/skill$ gcc stderr_redirect.c -o stderr_redirect
root@Mani:~/skill$ ./stderr_redirect
Error output redirected to error.txt
root@Mani:~/skill$
```

Report: The program parses stderr redirection and opens a file for storing error messages.
The standard error stream is redirected using dup2() with STDERR_FILENO.
Command failures are captured in the specified file for verification.
The program was tested successfully using invalid commands and verified error output.